

IFT3913 - Tâche 2 - Documentation

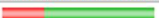
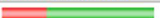
Islem Chokri

Yuan Qing Ji

Octobre 2025

Choix des classes

Nous avons choisi la classe Circle dans com.graphhopper.util.shapes. Cette classe est un bon candidat, car elle a des tests qui ne couvrent pas 100% du code : elle a 74% de couverture des instructions et 66% de couverture des branches.

GraphHopper Core > com.graphhopper.util.shapes												
com.graphhopper.util.shapes												
Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
BBox		71%		70%	31	79	31	129	7	28	0	1
Circle		74%		66%	26	51	16	69	6	14	0	1
Polygon		71%		66%	8	16	8	31	6	13	0	1
Total	387 of 1,432	72%	57 of 182	68%	65	146	55	229	19	55	0	3

Cette classe représente une aire géographique circulaire définie par une latitude et une longitude pour le centre, ainsi qu'un rayon en mètres.

Note : les index.html du jacoco et des pitests sont dans le répertoire « tache2-remise ».

Documentation des tests

Test 1

Nom: testContainsCoordinates()

Intention: vérifier si la fonction circle.contains(lat, lon) identifie correctement les coordonnées à l'intérieur ou à l'extérieur du cercle géographique

Motivation: utiliser des coordonnées simples autour de (0,0) pour faciliter le calcul et tester les points dans et hors du cercle

Oracle: les coordonnées sont à l'intérieur du cercle si la distance du point et de l'origine (0,0) est $\leq$ radiusInMeter

Test 2

Nom: testGeographicProperties()

Intention: vérifier que les valeurs de la latitude, de la longitude et du rayon sont set correctement initialisées et accessibles via les getters

Motivation: utiliser des coordonnées géographiques réelles pour assurer un comportement réel

Oracle: circle.getLat() retourne la latitude définie, circle.getLon() retourne la longitude définie, circle.radiusInMeter retournent le rayon défini et circle.contains retourne true pour les coordonnées correspondant au centre du cercle.

Test 3

Nom: testCustomDistanceCalcConstructor()

Intention: vérifier que Circle fonctionne avec un objet DistanceCalc personnalisé

Motivation: utiliser DistanceCalcEarth pour valider la compatibilité, car certains calculs dépendent du calculateur de distance

Oracle: le constructeur set les valeurs des champs correctement et les getters retournent les bonnes valeurs respectives

Test 4

Nom: testBasicBBoxIntersections()

Intention: vérifier le comportement de la fonction intersects(Bbox) pour différents cas de BBox à l'intérieur ou à l'extérieur du cercle

Motivation: assurer que circle.intersects fonctionne correctement selon la position relative du rectangle

Oracle: circle.intersects retourne true si le Bbox est entièrement ou partiellement à l'intérieur du cercle et circle.intersects retourne false si le Bbox est entièrement à l'extérieur du cercle

Test 5

Nom: testCircleRobustness()

Intention: vérifier la robustesse de la classe Circle avec de différentes tailles de cercles

Motivation: tester les résultats dans des scénarios avec des cercles très petits ou très grands afin de garantir le bon fonctionnement des méthodes principales

Oracle: *.contains() retourne true si le cercle contient son propre centre, *.contains() retourne false si le cercle ne contient pas le point, *.getBounds assure que les bornes ne sont jamais nulles et la méthode *.toString() retourne une représentation en string

Test 6

Nom: testDifferentCircleSizes()

Intention: vérifier le comportement de la méthode circle.contains() pour les cercles de différentes tailles

Motivation: assurer qu'un point appartient bel et bien au cercle selon sa taille

Oracle: un petit cercle contient son centre et ne contient pas les points à l'extérieur de son rayon, et un grand cercle contient son centre et ne contient pas les points à l'extérieur de son rayon

Test 7

Nom: testGettersAndToString()

Intention: vérifier le bon fonctionnement des getters et de la méthode ToString

Motivation: assurer que les données d'un cercle sont correctement accessibles et bien représentées sous forme textuelle

Oracle: getLat() et getLon() retournent les valeurs définies, et toString() contient la latitude, la longitude et le rayon du cercle

Tests supplémentaires pour améliorer la couverture des mutants

Test 8

Nom: testIntersectsBBoxAllBranches()

Intention: vérifier que intersects(Bbox) fonctionne correctement avec toutes les branches (cas limites)

Motivation: le test assure une couverture complète de toutes les branches logiques selon la position relative du cercle et du Bbox selon les conditions imbriquées de intersects(Bbox)

Oracle: intersects() retourne true si le Bbox est partiellement ou entièrement dans le cercle et retourne false si le Bbox est entièrement à l'extérieur du cercle

Test 9

Nom: testContainsBBoxEdgeCases()

Intention: vérifier que contains() fonctionne correctement si le Bbox est entièrement dans le cercle ou dépasse ses limites

Motivation: assurer de la bonne gestion des cas limites pour éliminer les faux positifs ou faux négatifs

Oracle: contains() retourne true si le Bbox est entièrement inclus et retourne false si le Bbox dépasse sur un coin, touche juste les bords ou est plus grand que le cercle

Test 10

Nom: testContainsCircleEdgeCases()

Intention: vérifier que contains() fonctionne correctement pour les cas où un cercle est dans un cercle principal

Motivation: assurer de la bonne gestion des cas limites pour éliminer les faux positifs ou faux négatifs

Oracle: contains() retourne true si le cercle est entièrement dans le cercle principal, si le cercle est tangent mais plus petit, et si le cercle a une distance : $\text{distance} + \text{rayonPetit} = \text{rayonGrand}$; contains() retourne false si le cercle dépasse légèrement le cercle principal, si le cercle est plus grand que le cercle principal et si le cercle a un même centre, mais avec un rayon plus grand

Test 11

Nom: testIntersectsPointListEdgeCases()

Intention: vérifier que intersects() fonctionne correctement avec les cas limites associés aux listes de points

Motivation: tester les comportements des cas limites pour les PointList vide, contenant un seul point à l'intérieur, contenant un seul point à l'extérieur, contenant un segment traversant le cercle sans que les points soient à l'intérieur et contenant un dernier point à l'extérieur

Oracle: intersects() retourne true pour le cas contenant un seul point à l'intérieur, contenant un segment traversant le cercle sans que les points soient à l'intérieur et contenant un dernier point à l'extérieur; intersects() retourne false pour le cas contenant un seul point à l'extérieur; intersects() throw une exception si le PointList est vide

Test 12

Nom: testEqualsAndHashCodeComprehensive()

Intention: vérifier le fonctionnement des méthodes equals et hashCode de la classe circle

Motivation: assurer du comportement des cas limites, et que le hashCode soit cohérent

Oracle: pour une comparaison avec soi-même, assertEquals retourne true; pour 2 cercles identiques, assertEquals() retourne true et les hashCode sont idem; pour les cercles différents, assertEquals() retourne false; pour les comparaisons avec null ou objet de type différent, assertEquals retourne false

Test 13

Nom: testConstructorAndBounds()

Intention: vérifier que le constructeur initialise les champs correctement et que la méthode getBounds() retourne un Bbox valide

Motivation: assurer qu'un objet Circle soit correctement initialisé avec DistanceCalc et que les bornes calculées correspondent au cercle

Oracle: getLat(), getLon() et radiusInMeter retournent les valeurs définies au constructeur, et getBounds retourne un Bbox non nul qui contient le centre du cercle

Pitest

Score de mutation avec les tests originaux

Pit Test Coverage Report

Package Summary

com.graphhopper.util.shapes

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	58% 40/69	23% 24/105	56% 24/43

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
Circle.java	58% 40/69	23% 24/105	56% 24/43

Report generated by [PIT](#) 1.15.0

Le score de mutation pour la classe Circle avec les tests originaux est de 23%, dont 24 mutants sur 105 ont été tués.

Score de mutation avec les nouveaux tests

Pit Test Coverage Report

Package Summary

com.graphhopper.util.shapes

Number of Classes	Line Coverage	Mutation Coverage	Test Strength
1	99% 68/69	48% 50/105	48% 50/104

Breakdown by Class

Name	Line Coverage	Mutation Coverage	Test Strength
Circle.java	99% 68/69	48% 50/105	48% 50/104

Report generated by [PIT](#) 1.15.0

Le score de mutation pour la classe Circle avec les tests ajoutés est de 48%, dont 50 mutants sur 105 ont été tués, soit 26 nouveaux mutants de tués.

Mutants détectés

En cliquant sur l'hyperlien Circle.java des pitests, on peut voir les mutants dans la section « Mutations ». En comparant ainsi les deux sections, on remarque de nouveaux mutants « KILLED » qui étaient soit « NO_COVERAGE » ou « SURVIVED »

Mutations	
50	1. replaced double return with 0.0d for com/graphhopper/util/shapes/Circle::getLat → NO_COVERAGE
54	1. replaced double return with 0.0d for com/graphhopper/util/shapes/Circle::getLon → NO_COVERAGE
59	1. removed conditional - replaced comparison check with false → KILLED
64	2. changed conditional boundary → SURVIVED
68	3. replaced boolean return with true for com/graphhopper/util/shapes/Circle::contains → SURVIVED
75	1. replaced return value with null for com/graphhopper/util/shapes/Circle::getBounds → NO_COVERAGE
80	1. replaced double return with 0.0d for com/graphhopper/util/shapes/Circle::normDist → SURVIVED
81	1. removed conditional - replaced equality check with false → SURVIVED
83	1. removed conditional - replaced equality check with false → SURVIVED
87	1. replaced boolean return with true for com/graphhopper/util/shapes/Circle::intersects → NO_COVERAGE
88	2. changed conditional boundary → NO_COVERAGE
89	3. removed conditional - replaced comparison check with false → NO_COVERAGE
91	1. removed conditional - replaced comparison check with false → KILLED
92	2. changed conditional boundary → KILLED
93	1. removed conditional - replaced equality check with false → KILLED
98	1. removed conditional - replaced comparison check with false → KILLED
103	2. changed conditional boundary → SURVIVED
104	1. replaced boolean return with false for com/graphhopper/util/shapes/Circle::intersects → KILLED
105	2. removed conditional - replaced comparison check with false → SURVIVED
107	1. changed conditional boundary → SURVIVED
108	2. removed conditional - replaced comparison check with false → SURVIVED
110	1. removed conditional - replaced comparison check with false → NO_COVERAGE
114	2. replaced boolean return with true for com/graphhopper/util/shapes/Circle::intersects → NO_COVERAGE
115	3. changed conditional boundary → NO_COVERAGE
116	1. replaced boolean return with true for com/graphhopper/util/shapes/Circle::intersects → KILLED
	2. changed conditional boundary → SURVIVED
	3. Replaced double subtraction with addition → KILLED
	4. removed conditional - replaced comparison check with false → KILLED
	1. changed conditional boundary → NO_COVERAGE
	2. removed conditional - replaced comparison check with false → NO_COVERAGE
	1. changed conditional boundary → NO_COVERAGE
	2. removed conditional - replaced comparison check with false → NO_COVERAGE
	1. removed conditional - replaced comparison check with false → NO_COVERAGE
	2. replaced boolean return with true for com/graphhopper/util/shapes/Circle::intersects → NO_COVERAGE

Figure 1 – Mutants avec les tests originaux

Mutations	
50	1. replaced double return with 0.0d for com/graphhopper/util/shapes/Circle::getLat → KILLED
54	1. replaced double return with 0.0d for com/graphhopper/util/shapes/Circle::getLon → KILLED
	1. removed conditional - replaced comparison check with false → KILLED
59	2. changed conditional boundary → SURVIVED
	3. replaced boolean return with true for com/graphhopper/util/shapes/Circle::contains → KILLED
64	1. replaced return value with null for com/graphhopper/util/shapes/Circle::getBounds → KILLED
68	1. replaced double return with 0.0d for com/graphhopper/util/shapes/Circle::normDist → KILLED
75	1. removed conditional - replaced equality check with false → KILLED
80	1. removed conditional - replaced equality check with false → KILLED
	1. replaced boolean return with true for com/graphhopper/util/shapes/Circle::intersects → KILLED
81	2. changed conditional boundary → SURVIVED
	3. removed conditional - replaced comparison check with false → KILLED
83	1. removed conditional - replaced comparison check with false → KILLED
	2. changed conditional boundary → KILLED
87	1. removed conditional - replaced equality check with false → KILLED
88	1. removed conditional - replaced comparison check with false → KILLED
	2. changed conditional boundary → SURVIVED
89	1. replaced boolean return with false for com/graphhopper/util/shapes/Circle::intersects → KILLED
	1. removed conditional - replaced comparison check with false → SURVIVED
91	2. changed conditional boundary → SURVIVED
	3. removed conditional - replaced equality check with false → KILLED
	4. Replaced integer addition with subtraction → KILLED
92	1. removed conditional - replaced comparison check with false → KILLED
	2. changed conditional boundary → SURVIVED
93	1. replaced boolean return with false for com/graphhopper/util/shapes/Circle::intersects → KILLED
98	1. replaced boolean return with true for com/graphhopper/util/shapes/Circle::intersects → KILLED
103	1. removed conditional - replaced comparison check with false → KILLED
	2. changed conditional boundary → SURVIVED
104	1. removed conditional - replaced comparison check with false → SURVIVED
	2. changed conditional boundary → SURVIVED
105	1. replaced boolean return with true for com/graphhopper/util/shapes/Circle::intersects → SURVIVED
	2. changed conditional boundary → SURVIVED
	3. removed conditional - replaced comparison check with false → KILLED
107	1. changed conditional boundary → SURVIVED
	2. removed conditional - replaced comparison check with false → SURVIVED
108	1. removed conditional - replaced comparison check with false → KILLED
	2. replaced boolean return with true for com/graphhopper/util/shapes/Circle::intersects → SURVIVED
	3. changed conditional boundary → SURVIVED
110	1. replaced boolean return with true for com/graphhopper/util/shapes/Circle::intersects → KILLED
	2. changed conditional boundary → SURVIVED
	3. Replaced double subtraction with addition → KILLED
	4. removed conditional - replaced comparison check with false → KILLED
114	1. changed conditional boundary → SURVIVED
	2. removed conditional - replaced comparison check with false → SURVIVED
115	1. changed conditional boundary → SURVIVED
	2. removed conditional - replaced comparison check with false → SURVIVED
116	1. removed conditional - replaced comparison check with false → KILLED
	2. replaced boolean return with true for com/graphhopper/util/shapes/Circle::intersects → KILLED
	3. changed conditional boundary → SURVIVED
118	1. removed conditional - replaced comparison check with false → SURVIVED
	2. changed conditional boundary → SURVIVED

Figure 2 – Mutants avec les nouveaux tests

En comparant la Figure 1 avec la Figure 2, nous pouvons observer que les mutants des lignes 50, 54, 64, 68, 75, 80, etc. ont été tués par les nouveaux tests. En survolant ma souris sur les mutants tués, je peux voir quelle méthode l'a tué. Par exemple, pour le mutant de la ligne 50, notre test « testConstructorAndBounds() » l'a tué (Figure 3), car le test vérifie la création correcte du cercle et de ses bornes, assurant ainsi que getLat() retourne exactement la bonne valeur pour le champ (Figure 4). De cette manière, le test assure que les getters retournent la bonne valeur de latitude, par exemple, dans le cas où le mutant modifie la valeur du champ ou utilise la valeur d'un autre champ, comme la longitude, au lieu de la valeur de la latitude.

50	1. re	Location : getLat
54	1. re	Killed by : com.graphhopper.util.shapes.CircleTest.[engine:junit-
	1. re	jupiter]/[class:com.graphhopper.util.shapes.CircleTest]/[method:testConstructorAndBounds()
59	2. ch	]
	3. replaced boolean return with true for com/graphhopper/util/shapes/Circle::contains → KILLED	

Figure 3 – Mutant ligne 50 tué par testConstructorAndBounds()


```

48
49     public double getLat() {
50 1      return lat;
51     }
52

```

Figure 4 – Effet du test sur le mutant de la ligne 50

Le mutant de la ligne 75 a été tué par notre test « testIntersectsPointListEdgeCases() » (Figure 5), car le test évalue la condition où le PointList est vide. Ce test a été conçu pour forcer l'exécution des cas limites, et dans ce cas, lancé une exception quand PointList est vide (len == 0). Un mutant peut inverser la condition booléenne ou l'opérateur ($\leq$, $\geq$, $=$, etc.), et notre test couvre toutes les conditions logiques associées aux cas limites, permettant ainsi d'identifier ce mutant.

```

64 1. replaced return value with null for com.graphhopper.util.shapes.Circle::getBounds → KILLED
68 1. re 1
75 1. re Location : intersects
80 1. re Killed by : com.graphhopper.util.shapes.CircleTest.[engine:junit-
1. re jupiter]/[class:com.graphhopper.util.shapes.CircleTest]/[method:testIntersectsPointListEdg
81 2. ch eCases()]
3. removed conditional - replaced comparison check with false → KILLED
~ 1. removed conditional - replaced comparison check with false → KILLED

```

Figure 5 – Mutant ligne 75 tué par testIntersectsPointListEdgeCases()

```

72     public boolean intersects(PointList pointList) {
73         // similar code to LocationIndexTree.checkAdjacent
74         int len = pointList.size();
75 1      if (len == 0)
76         throw new IllegalArgumentException("PointList must not be empty");
77

```

Figure 6 – Effet du test sur le mutant de la ligne 50

D'après les observations, nous pouvons conclure que nos tests sont efficaces à détecter et tuer de nouveaux mutants

JavaFaker

Test

Nom: testCircleWithFakerData()

Intention: vérifier du bon fonctionnement de la classe Circle avec des valeurs générées dynamiquement à l'aide de la librairie JavaFaker

Motivation: simuler de différents cercles pour tester la robustesse du constructeur et des méthodes avec des valeurs dynamiques

Oracle: getLat() et getLon() retournent les valeurs de latitude et de longitude initialisées, contains() retourne true si le point est à l'intérieur du cercle, toString() retourne une représentation string valide du cercle, le test affiche le contexte géographique (ville et pays) généré par JavaFaker